Due Date: Wednesday, October 21 at 11:59 PM

Your task: Create a desktop app that runs in the console. You'll create an interactive "Guess the Number" game, which picks a number between 1 and 10 and compares it to the player's guess. The goal of this activity is to practice the creation of Node.js projects from scratch, get familiar with a few native modules and external packages, and have hands-on experience with GitHub.

Project Requirements:

1. All the following instructions must be created in the terminal, console or windows power shell. Evidence must be submitted along with your solution. A screenshot will suffice.
2. Create the project and JS file
3. Initialize `npm` at the root of this project
4. Install these packages: `prompts` and `chalk`.
5. **Commit** your changes to GitHub.

App Requirements:

1. Verify the configuration file supports ESM.
2. Display the game title, instructions, and rules, using the external package `chalk`.
3. The guessing game must generate a random secret number between 1 and 10. To do so, use the `randomInt` function available in the native module `crypto`.
4. Create a constant to hold the maximum number of attempts.
5. Use the external package `prompts` to create an interactive game. Ask the player to enter a number between 1 and 10.
6. Second **commit** to GitHub.
7. Validate player's input. To do us use the native `Error` class to:
    a. Reject empty input. Use the string `trim()` function.
    b. Reject non-numeric input and decimal values. Use the `isInteger()` function from the `Number` class to determine whether the passed value is a whole integer.
    c. Reject negative and zero values
    d. Reject numbers above the maximum threshold.
8. Use `try… catch` block statements to handle validation errors and use `chalk` to display messages in red without crashing the game.

9. Compare valid guesses with the secret number and display hints, such as: too low, or too high.
10. Use `chalk` to pick a color and display a message when the player guesses the secret number.
11. Use `chalk` again to select a different color to display a message when the player runs out of attempts.
12. Export all output messages to a file, using the native module `fs`.
13. Last **commit** to GitHub.

Sample Output

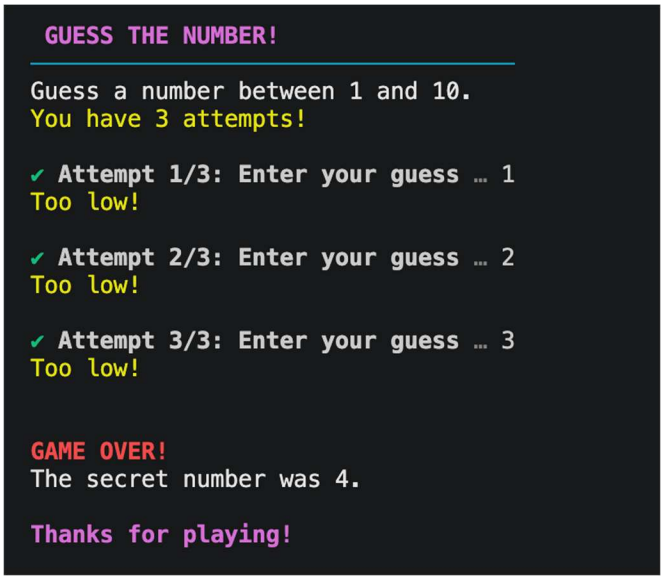


Fig 1. Gameplay demonstration

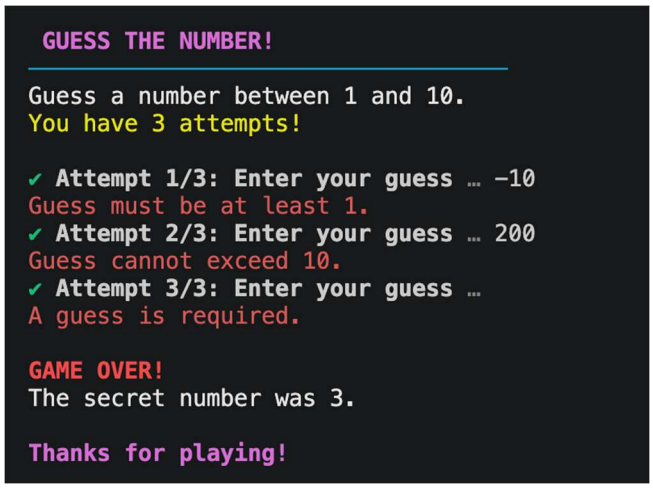


Fig 2. Error validation kicking in.

Submission: Your solution must be hosted in GitHub, and **your repository should contain multiple commits** (see grading criteria) demonstrating your progress throughout the assignment. Use the Brightspace submission tools to submit a single text file containing the link to your GitHub repository.

Grading criteria:

-25: late
-10: missing evidence of implementation of project requirements.
-10: Chalk package implementation
-10: Prompts package implementation
-10: Crypto native module implementation
-10: Error native module implementation
-2 for each missing validation scenario (4 input validations in total)
-7: Error handling is missing
-4: Repetition statement is missing
-4: Conditional statement that verifies player's input vs secret number is missing

At minimum, three commits are required:
-9 First commit: node project with dependencies.
-9. Second commit: messaging, such as game title and input.
-9. Third commit: game implementation